Universidad de Valladolid

Ingeniería Informática

Informe Proyecto Servicios y Sistemas Web

Proyecto web tipo Fantasy sobre la liga escocesa

Grupo 12 de Sistemas y Sistemas Web
27 de mayo de 2026

Autores
Ángel Antolín Caramazana
Marcos Gómez Medina
Jorge Hernández Sánchez
Luis Antón Sanz

Índice

1. Presentación del Equipo y Estimacion de Trabajo **2**

2. Introducción **2**
2.1. Estructura del proyecto: 3
2.2. Desglose del Sistema de Puntuación 3
2.3. Sistema de Bonificación por Química 4

3. Usuarios Objetivo **4**

4. Especificación de Requisitos **4**
4.1. Requisitos Funcionales (RF) 4
4.2. Requisitos No Funcionales (RNF) 5

5. Modelo del Dominio **6**

6. Mapa del Sitio Web **6**

7. Estructura de la Base de Datos **7**
7.1. Orden de Ejecución de los Scripts 7

8. Descripción de la Funcionalidad Implementada y Justificación **8**
8.1. Funcionalidad básica de la aplicación 8
8.2. Arquitectura basada en el Patrón Modelo-Vista-Controlador (MVC) 8
8.3. Consumo Unificado de Datos Reales mediante API Externa 9
8.4. Seguridad Criptográfica 9
8.5. Manejo Global y Declarativo de Errores 9

9. Conclusiones **9**

10. Declaración de Uso de Inteligencia Artificial **10**

1 Presentación del Equipo y Estimacion de Trabajo

Este documento detalla la realización de la práctica de Servicios y Sistemas Web por parte del Grupo 12, perteneciente al tercer curso del Grado de Ingeniería Informática en la Universidad de Valladolid (UVa). El equipo está compuesto por los siguientes alumnos:

- Marcos Gómez Medina 25 %: Realizacion del pdf de la entrega y parte de implementacion de puntaje
- Ángel Antolín Caramazana 25 %: implementacion del mercado de jugadores y cambio de alineacion y jugadores en el equipo de los usuarios
- Jorge Hernández Sánchez 25 %: implementacion de la API para obtener las puntuaciones y actuaciones de partidos
- Luis Antón Sanz 25 %: parte de implementacion del puntaje e implementacion de observar equipos de usuarios rivales

2 Introducción

Para esta práctica se ha realizado una aplicación web de tipo "Fantasy". Se trata de una plataforma interactiva donde, en función del desempeño deportivo real en diferentes disciplinas, se aplican una serie de puntos formando una liga para jugar con amigos de forma competitiva y sana. El usuario con más puntos acumulados al final de la temporada regular se proclama como el ganador del torneo.

En nuestro caso, el sistema desarrollado se denomina **"Fant11sy"**, una aplicación basada en la Primera División de la Liga Escocesa de Fútbol, más conocida como *Scottish Premiership*.

El funcionamiento del juego es directo e intuitivo: antes del inicio de cada jornada futbolística, los usuarios eligen sabiamente su alineación titular compuesta por 11 jugadores. En función del desempeño deportivo de la vida real de los futbolistas seleccionados en sus respectivos partidos de liga, el sistema asignará puntuaciones automatizadas.

2.1 Estructura del proyecto:

- **Paquete fant11sy:** Encontramos los distintos archivos java, ya sea Servlets, implementacion de la API, DAOS, clases JavaBean, etc.
- **Paquete resources:** Encontramos el archivo del token de la API y los distintos scripts sql.
- **Paquete webapp:** Se encunetran los archivos JSP, sus estilos en CSS y las imágenes utilizadas en las páginas así como el archivo de configuración web.xml

```
fant11sy-1.0-SNAPSHOT [main]
└── src
    └── main
        ├── java
        │   └── com
        │       └── uva
        │           └── fant11sy
        │               ├── api
        │               ├── daos
        │               ├── db
        │               ├── modelo
        │               ├── servlets
        │               └── util
        ├── resources
        │   ├── META-INF
        │   │   └── config.properties
        │   └── sql
        │       ├── fantasy11.sql
        │       ├── partidosPrueba.sql
        │       ├── pruebaEquipo.sql
        │       ├── pruebaUsuarios.sql
        │       └── simulacionPuntuacion.sql
        └── webapp
            ├── META-INF
            ├── WEB-INF
            ├── images
            └── styles
```

2.2 Desglose del Sistema de Puntuación

Las estadísticas recolectadas se traducen en puntos positivos o negativos para el equipo del usuario bajo el siguiente criterio matemático:

- **Gol anotado:**
  - Portero o defensa: +6 puntos
  - Mediocentro: +5 puntos
  - Delantero: +4 puntos
- **Asistencia de gol:** +3 puntos.

- **Goles en propia:** -2 puntos
- **Goles en contra** (aplicable exclusivamente a porteros y defensas): -1 punto por cada 2 goles en contra.
- **Portería a cero** (aplicable exclusivamente a porteros y defensas): +3 puntos.
- **Tarjeta amarilla:** -2 puntos.
- **Tarjeta roja:** -4 puntos.
- **Minutos jugados en el partido:**
  - Disputar al menos 30 minutos: +1 punto.
  - Disputar al menos 60 minutos: +2 puntos.
  - Disputar 90 minutos completos: +3 puntos.

2.3 Sistema de Bonificación por Química

Adicionalmente, se ha implantado un sistema de bonificaciones basado en la "química" táctica. El objetivo de este mecanismo es incentivar a los usuarios a planificar plantillas estratégicas y cohesionadas utilizando jugadores de un mismo país o de un mismo club real, aumentando así la variedad competitiva:

- **Bonus por Club:** +2 puntos por cada pareja de jugadores que pertenezcan al mismo club real.
- **Bonus por País:** +1 punto por cada pareja de jugadores que compartan la misma nacionalidad.

Estos puntos de bonificación por alineación son completamente independientes de los eventos reales del fin de semana y se calculan nativamente en el backend al procesar la jornada.

3 Usuarios Objetivo

Esta aplicación se encuentra dirigida a aquellas personas que disfruten del fútbol y les guste la competividad con sus amigos de una manera sana y divertida.

Buscamos atraer usuarios aficionados de ligas internacionales y que busquen una aplicación de fácil uso y ampliamente intuitiva de manejar, pues el principal rechazo de este tipo de aplicaciones es su complejidad por sus sistema de mercado.

4 Especificación de Requisitos

4.1 Requisitos Funcionales (RF)

RF-1: Inicio de sesión mediante proceso de login Se permite al usuario iniciar sesion en la aplicación mediante un nombre de usuario único y una contraseña. El sistema cifra la contraseña en la base de datos de forma segura.

RF-2: Gestión y selección de alineaciones El usuario puede gestionar su plantilla y realizar fichajes de jugadores para cubrir los slots de su once inicial (formación fija 4-3-3), pudiendo realizar modificaciones siempre y cuando se hagan antes del comienzo de la jornada real.

RF-3: Restricción estricta de plantilla completa Es mandatorio disponer de exactamente 11 jugadores en el once inicial para poder puntuar en la jornada. En caso contrario, el sistema aplicará penalización automática asignando cero puntos globales al usuario en esa fecha.

RF-4: Simulación de jornadas a partir de datos reales (API) El sistema consume datos reales de rendimiento deportivo mediante la API externa *Sportmonks*, lo que permite procesar marcadores de partidos y las actuaciones individuales de los futbolistas.

RF-5: Consulta de clasificaciones El usuario dispone de una sección donde consultar la tabla de clasificación con el resto de rivales de la liga, permitiendo filtrar los resultados de manera global (acumulada) o específica por cada jornada individual.

RF-6: Desglose de bonificaciones informativas En la pantalla del equipo se muestra dinámicamente un desglose sobre el conjunto de bonificaciones por país o club acumuladas por la alineación actual para orientar al usuario en su estrategia.

RF-7: Visualización de alineaciones rivales Con el fin de fomentar la competitividad y el análisis de estrategias de juego, se permite a los usuarios examinar en modo lectura el once inicial configurado por sus rivales de liga desde la tabla de clasificación.

RF-8: Registro de nuevos usuarios Se permite a nuevos usuarios registrarse en la aplicacion con un nombre y un usuario único.

RF-9: Consulta de calendario La aplicación muestra el calendario de partidos y los resultados de estos agrupados por jornadas permitiendo consultar cualquier jornada.

RF-10: Consulta de clubes Desde el calendario de partidos se debe poder acceder al listado de jugadores de un club.

4.2 Requisitos No Funcionales (RNF)

RNF-1: Rendimiento y tiempo de respuesta El tiempo de respuesta en los procesos críticos como la autenticación (login) o la carga del panel de alineación debe ser inferior a 10 segundos bajo condiciones normales de red.

RNF-2: Gestión de sesión y persistencia de memoria El servidor debe mantener de forma segura el estado de la sesión del usuario autenticado, recordando de manera robusta los jugadores asignados a su equipo sin pérdidas de información.

RNF-3: Tolerancia a fallos y robustez El sistema debe ser capaz de recuperarse de fallos inesperados en la capa de datos o fallos de red con la API externa, evitando caídas totales del servicio e informando al usuario adecuadamente.

5 Modelo del Dominio

El modelo del dominio detalla las entidades conceptuales del sistema (Usuario, Equipo, Jugador, Partido, Club, Actuación) y sus relaciones lógicas. Se encuentra actualizado respecto al modelo de dominio original que se presentó al principio de este proyecto.

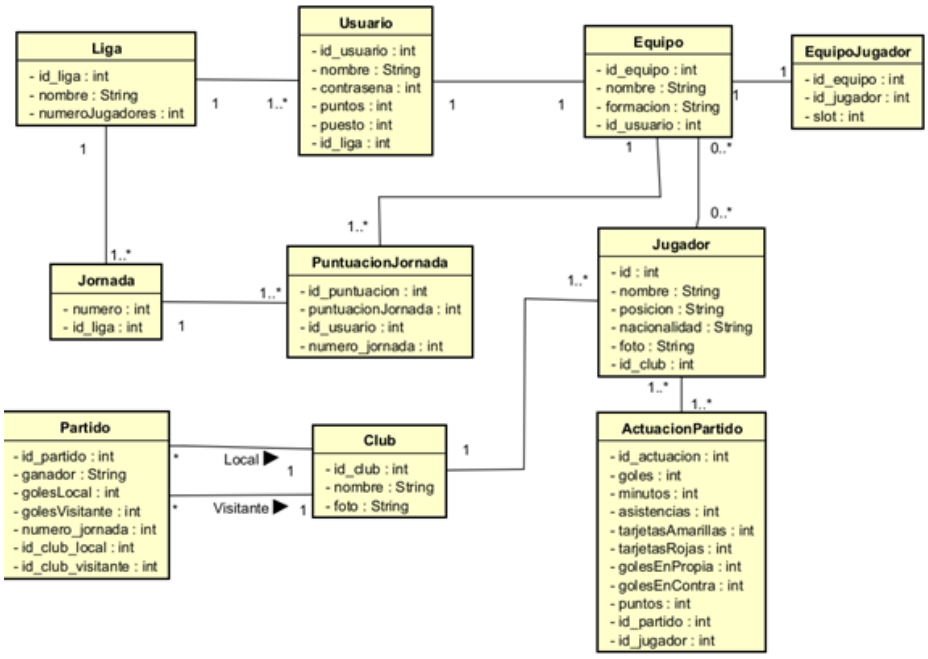


Figura 1: Diagrama del Modelo del Dominio de Fant11sy

6 Mapa del Sitio Web

El mapa del sitio web con el que se produce el flujo de trabajo es el siguiente.

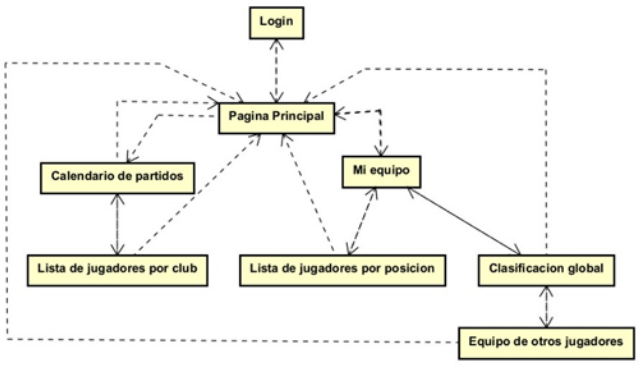


Figura 2: Mapa del Sitio Web

7 Estructura de la Base de Datos

Se ha diseñado una base de datos relacional robusta gestionada a través del motor **MariaDB**. Se usan scripts pra la defínción de tablas, restricciones de integridad y claves foráneas.

7.1 Orden de Ejecución de los Scripts

Para inicializar correctamente el entorno de base de datos y poblar el sistema con datos de prueba estables para la demostración, se debe seguir el siguiente orden estricto de ejecución en el servidor de base de datos:

1. `fantasy11.sql`**:** Script principal que crea la estructura de la base de datos vaciando previamente cualquier rastro anterior y generando las tablas e índices correspondientes.

2. `pruebaUsuarios.sql`**:** Registra los perfiles de los usuarios competidores (integrantes del grupo de desarrollo) junto con sus equipos base y las puntuaciones iniciales de las jornadas previas.

3. `Iniciar el proyecto y añadir /cargar-api a la URL`**:** Utiliza la API para cargar en la base de datos los jugadores y sus respectivos clubes.

4. `partidosPrueba.sql`**:** Rellena el calendario de partidos con una serie de partidos para que se pueda mostrar.

5. `simulaPuntuaciones.sql`**:** Utilizado en la presentacion para motrar como funciona el sistema de puntuacion, carga resultados de partidos y actuaciones de unos jugadores predefinidos para que poder comprobar como se actualiza la clasificacion.

6. `Añadir /ActualizarJornadaServlet a la URL`**:** En caso de que se esté haciendo una prueba con el anterior script, en vez de coger datos de la API directamente, se deberá comentar la línea que llama a la API de este Servlet. Se encargará de coger las actuaciones de los jugadores de la última jornada y actualizar las clasificaciones.

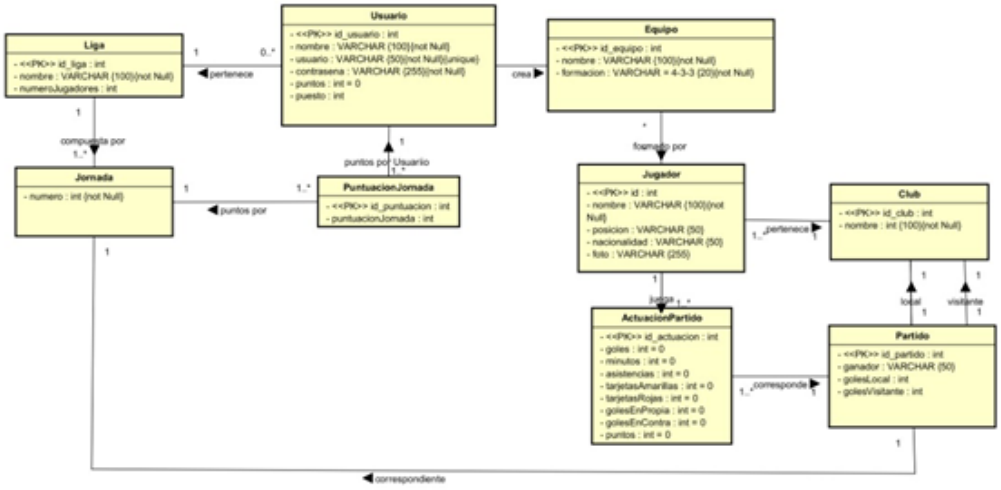


Figura 3: Diagrama Entidad-Relación de la Base de Datos Relacional

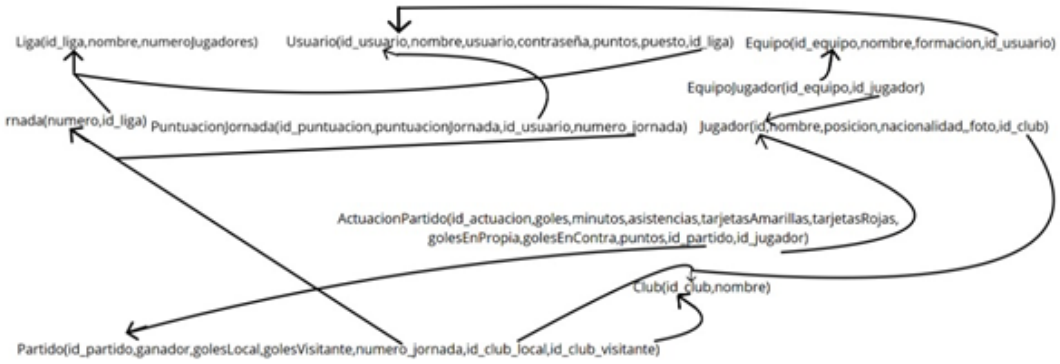


Figura 4: Diagrama Relacional

8 Descripción de la Funcionalidad Implementada y Justificación

Durante el ciclo de desarrollo, la funcionalidad se refinó para alcanzar un objetivo realista y robusto adaptado a las necesidades de entrega.

8.1 Funcionalidad básica de la aplicación

Un usuario puede registrarse e iniciar sesión en la aplicación, eso le asignará un equipo y le unirá a la liga.

Una vez en la aplicación se puede consultar el calendario de partidos completo por jornadas o acceder a la página de tu equipo.

En el equipo tomamos la decisión de eliminar el mercado propio de este tipo de aplicaciones con manejo de dinero y dejarlo en un sistema simple en el que cada usuario debe montar un equipo de 11 jugadores que el quiera, buscando maximizar sus bonus y puntuación en base a los resultados que estén dando os equipos reales.

Cada jornada actualizará las actuaciones de los jugadores sumando puntos a los usuarios que tuvieran un 11 inicial válido, actualizando la clasificación global y de esa jornada que podrá ser consultada en todo momento por los usuarios.

Comentario sobre la funcionalidad de la API: Se ha implementado la API completamente, obtenemos todos los partidos, clubes y jugadores y cada fin de semana tras una jornada podemos obtener de esta misma el resultado de los partidos y las actuaciones de cada jugador. Sin embargo, en fecha de entrega de este trabajo, la liga escocesa ha finalizado, por lo que todas las pruebas de obtener resultados y puntuaciones hemos tenido que hacerlas con scripts sql y no con la propia API.

La implementación está ahí y se podría usar en un entorno real, pero por este motivo en el Servlet **ActualizarJornadaServlet**, dejamos comentado la parte de obtener los datos de la API durante la presentación, pues introduciremos esos datos manualmente con un script externo y este Servlet solo hará el cálculo de puntos.

8.2 Arquitectura basada en el Patrón Modelo-Vista-Controlador (MVC)

Se ha seguido una separación estricta de responsabilidades:

- **Modelo:** Representado por clases JavaBean puras (paquete `modelo`), utilizadas por los Servlets y nuestra capa de persistencia utilizando el patrón DAO, representan nuestros

objetos del dominio y nuestra forma de comunicar los Servlets con los datos obtenidos por los DAOS de la BD.

- **Controlador:** Implementado mediante Servlets (paquete `servlets`). Los servlets interceptan las peticiones HTTP, validan las sesiones, invocan los servicios de datos necesarios y preparan los atributos requeridos por la vista.
- **Vista:** Páginas dinámicas JSP unificadas mediante hojas de estilo CSS. Las vistas no realizan lógica de negocio ni consultas a base de datos; simplemente renderizan la información provista por el servlet mediante etiquetas de expresión de manera limpia.

8.3 Consumo Unificado de Datos Reales mediante API Externa

La integración con la API de *Sportmonks* representa un pilar del proyecto. Para optimizar el rendimiento y reducir las peticiones HTTP concurrentes. Nos permitió agilizar el trabajo al evitar necesitar crear scripts para información de jugadores o clubes.

Cuenta con 3 clases:

- `ActualizarResultadosAPI.java:` Recoge los resultados de los partidos y las actuaciones de los jugadores en la fecha indicada.
- `CargarEquipos.java:` Obtiene la informacion de todos los clubes y jugadores de la liga.
- `CargarPartidosAPI.java:` Realiza la carga inicial de partidos rellenando todas las jornadas con partidos que serán jugados a lo largo de la liga.

Por motivos de seguridad y de acuerdo con la corrección del profesor en el prototipo, el token criptográfico de la API no se encuentra escrito directamente en el código fuente (*hardcoded*), sino que se encapsula en la clase `Config.java` leyéndose desde un archivo externo aislado denominado `config.properties` situado en el ClassPath.

8.4 Seguridad Criptográfica

El sistema de registro y login (`RegistroServlet` y `LoginServlet`) no almacena las contraseñas de los usuarios en texto plano dentro de la base de datos. Se ha desarrollado la clase de utilidad `Seguridad.java` que aplica el algoritmo de hashing seguro **SHA-256** en combinación con representaciones hexadecimales, garantizando la privacidad de las credenciales frente a posibles accesos no autorizados a las tablas.

8.5 Manejo Global y Declarativo de Errores

Siguiendo los apuntes, creamos una página centralizada de errores que recogiera las posibles excepciones de acceso a la base de datos o errores con la API y redirigiera allí a los usuarios.

9 Conclusiones

Durante todo este proceso se ha podido poner en práctica los diferentes conocimientos aprendidos en las sesiones teóricas y prácticas, permitiéndonos al grupo obtener una comprensión mucho más completa y estructurada a la par que se ha fomentado el trabajo en grupo con una repartición de trabajo estructurada y equitativa.

Por otro lado, la realización de una aplicación orientada a muchos de los gustos de los integrantes ha permitido un entorno de trabajo agradable.

En cuanto a la realización práctica, hemos podido comprender el uso básico de html, acompañado de una capa de estilización como es el CSS, siguiendo con Servlets y servidores, conexion de una API con la base de datos y la realización de scripts para pruebas de aceptación.

10 Declaración de Uso de Inteligencia Artificial

Herramientas Utilizadas: GeminiAI

Propósito del Uso: Depuración de código, sugerencias de implementación, creación de scripts de pruebas de aceptación, revisión y reescritura del pdf y generacion de imágenes para su uso por defecto en jugadores y escudos.

Alcance del Uso: proyecto de NetBeans, implementación de la API y pdf de entrega.

Ejemplos de Prompts Utilizados:

- *“ Te adjunto el siguiente codigo que tiene un error que es...”*
- *“Necesito implementar un Servlet que recoja los datos de los clubes y los muestre en el calendario”*
- *“Generame un script que introduzca una serie de partidos de prueba en varias jornadas...”*
- *“Revisa este documento y reescribelo para que tenga un tono más técnico acorde al tema”*
- *“Dame una imagen de una silueta para poner a un jugador por defecto”*